Systèmes embarqués et Internet des objets

Informatique — Licence 3 (L3)

Cours INF-L3-20

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Programmer le monde physique : microcontrôleurs Arduino et ESP32, capteurs et actionneurs, protocoles IoT (Wi-Fi, Bluetooth, LoRa, MQTT), collecte de données et sécurité des objets connectés, jusqu'à un projet complet de station météo ou d'arrosage automatique.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Microcontrôleurs et cartes de prototypage : Arduino
• Leçon 2 — Capteurs et actionneurs : lire et agir sur le monde physique
• Leçon 3 — Protocoles de communication pour l'IoT : initiation
• Leçon 4 — Programmer un objet connecté simple
• Leçon 5 — Collecte et affichage des données de capteurs
• Leçon 6 — Sécurité des objets connectés
• Leçon 7 — Projet : station météo ou système d'arrosage connecté
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Un réfrigérateur qui signale une porte mal fermée, un compteur qui transmet tout seul ses relevés, une serre qui arrose ses plants quand la terre devient sèche : derrière chacun de ces objets se trouve un système embarqué, c'est-à-dire un petit système informatique intégré à l'objet et dédié à une tâche précise. Ce cours t'apprend à construire de tels objets, de la première LED qui clignote jusqu'à la station météo connectée qui envoie ses mesures sur le réseau. C'est le point de rencontre entre l'informatique et le monde physique : ton code ne manipule plus seulement des données, il allume, mesure, arrose et alerte. Ces compétences sont au cœur de l'Internet des objets (IoT), un domaine qui recrute dans l'agriculture connectée, la domotique, la santé et l'industrie.

La démarche est entièrement pratique. Nous travaillerons avec les cartes Arduino Uno et ESP32, bon marché et documentées par une immense communauté : un microcontrôleur, quelques capteurs (température, lumière, humidité du sol, distance), des actionneurs (LED, relais, servomoteur) et une plaque d'essai suffisent pour tous les exemples et tous les travaux pratiques du cours. Chaque leçon suit le même rythme : comprendre la notion, câbler le montage, écrire le programme — le « sketch » —, puis vérifier le résultat sur le moniteur série. Les protocoles de communication (Wi-Fi, Bluetooth, LoRa et surtout MQTT, le protocole de messagerie léger de l'IoT) seront introduits au moment où nous en aurons besoin : pour faire dialoguer nos objets entre eux et avec un serveur.

Ce cours suppose que tu sais déjà programmer (variables, conditions, boucles et fonctions, vus en L1 et L2) et que tu connais les notions d'électricité du cours de L1 : tension, courant, résistance et loi d'Ohm. Aucune soudure n'est nécessaire : tous les montages se font sur plaque d'essai avec des fils de liaison. Deux avertissements avant de commencer : on ne branche jamais un montage alimenté, on vérifie toujours le câblage avant de mettre sous tension ; et un capteur ne donne jamais « la vérité », seulement une mesure qu'il faut savoir convertir, calibrer et parfois moyenner. Le projet final — station météo connectée ou arrosage automatique — réunira toutes ces compétences dans un objet complet, testé et documenté.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Décrire l'architecture d'un système embarqué et le rôle d'un microcontrôleur.
- Câbler et programmer des capteurs et des actionneurs sur une carte Arduino.
- Convertir une mesure analogique brute en grandeur physique (tension, température, distance).
- Comparer les protocoles IoT (Wi-Fi, Bluetooth, LoRa, MQTT, CoAP) et choisir selon la portée, le débit et la consommation.
- Programmer un objet connecté qui publie ses mesures via MQTT.
- Collecter, horodater, enregistrer et afficher des données de capteurs.
- Appliquer les règles de base de la sécurité des objets connectés et mener un projet embarqué complet et documenté.

Leçon 1 — Microcontrôleurs et cartes de prototypage : Arduino

Un système embarqué est un système informatique spécialisé, intégré dans un objet pour accomplir une tâche déterminée : gérer l'injection d'un moteur, afficher la température d'un four, compter les pas d'un bracelet connecté. Contrairement à un ordinateur personnel, qui doit tout savoir faire, il ne fait qu'une chose, mais il la fait en permanence, souvent en temps réel et avec très peu d'énergie. Son cerveau est le microcontrôleur : une puce unique qui intègre un processeur, la mémoire programme (flash, qui conserve le programme hors tension), la mémoire de données (SRAM, volatile), une petite mémoire permanente réinscriptible (EEPROM) et des périphériques d'entrée/sortie pour dialoguer avec des capteurs et des actionneurs.

La carte Arduino Uno, la plus utilisée pour apprendre, est construite autour du microcontrôleur ATmega328P cadencé à 16 MHz, avec 32 Ko de mémoire flash, 2 Ko de SRAM et 1 Ko d'EEPROM. Elle fonctionne en logique 5 V : une broche à l'état haut (HIGH) délivre 5 V, à l'état bas (LOW) 0 V. Elle offre 14 broches numériques D0 à D13 (dont 6, marquées ~, peuvent produire un signal PWM) et 6 entrées analogiques A0 à A5. On la programme et on l'alimente par son connecteur USB ; une alimentation externe de 7 à 12 V est possible pour un montage autonome. D'autres cartes complètent la famille : la Nano, compacte, utilise le même microcontrôleur ; la Mega offre davantage de broches et de mémoire ; l'ESP32, que nous utiliserons en leçon 4, fonctionne en 3,3 V et intègre directement le Wi-Fi et le Bluetooth.

On programme la carte depuis l'environnement de développement Arduino (l'IDE) installé sur un ordinateur. Le programme, appelé sketch, est écrit en C/C++ simplifié et contient toujours deux fonctions : setup(), exécutée une seule fois au démarrage, sert à configurer (par exemple déclarer une broche en sortie) ; loop() est ensuite répétée indéfiniment, c'est là que vit le comportement de l'objet. L'IDE compile le sketch, le téléverse par USB, et son moniteur série affiche les messages que la carte envoie : c'est notre principal outil de mise au point. Une vaste bibliothèque de code réutilisable, maintenue par la communauté, évite de tout réécrire pour chaque capteur. Le premier programme de tout apprenti, l'équivalent du « Bonjour le monde », consiste à faire clignoter une LED.

L'essentiel
Un microcontrôleur = processeur + mémoires + entrées/sorties sur une seule puce. La Uno (ATmega328P) travaille en 5 V, offre 14 broches numériques et 6 entrées analogiques. Tout sketch Arduino = setup() (une fois) puis loop() (en boucle).

Exemple — Faire clignoter la LED intégrée (broche 13)

```
// Leçon 1 — Faire clignoter la LED intégrée de la carte Uno
// La LED intégrée est reliée à la broche numérique 13 (D13).
// LED externe : anode (+) -> résistance 220 ohms -> broche D8
// cathode (-) -> GND. La résistance limite le courant.

void setup() {
pinMode(13, OUTPUT); // la broche 13 devient une SORTIE
}

void loop() {
digitalWrite(13, HIGH); // HIGH = 5 V : la LED s'allume
delay(1000); // pause de 1000 ms (1 seconde)
digitalWrite(13, LOW); // LOW = 0 V : la LED s'éteint
delay(1000); // pause de 1 seconde, puis on recommence
}
```

À retenir

- Un microcontrôleur intègre processeur, mémoires et entrées/sorties sur une seule puce : c'est un ordinateur miniature dédié à une tâche.
- Sur la Uno, les niveaux logiques sont 0 V (LOW) et 5 V (HIGH) ; les broches marquées ~ peuvent produire un signal PWM.
- Tout sketch Arduino contient setup() (une fois) et loop() (en boucle) ; le téléversement se fait par USB depuis l'IDE.

Exercice

On veut faire clignoter deux LED externes, branchées sur les broches 8 et 9, en alternance : quand l'une est allumée, l'autre est éteinte, et l'échange a lieu toutes les 500 ms. Écris le sketch complet et décris le câblage de chaque LED en précisant le rôle de la résistance.

✔ Corrigé
Câblage : pour chaque LED, l'anode (la longue patte) va vers sa broche (8 ou 9) à travers une résistance de 220 Ω, et la cathode (patte courte) vers GND. La résistance limite le courant qui traverse la LED : sans elle, la LED et la broche seraient traversées par un courant excessif et pourraient être détruites. Sketch : void setup() { pinMode(8, OUTPUT); pinMode(9, OUTPUT); } void loop() { digitalWrite(8, HIGH); digitalWrite(9, LOW); delay(500); digitalWrite(8, LOW); digitalWrite(9, HIGH); delay(500); }. Remarque : la période complète du clignotement vaut 500 + 500 = 1 000 ms, soit 1 Hz — à chaque tour de loop(), chaque LED a été allumée puis éteinte une fois.

Leçon 2 — Capteurs et actionneurs : lire et agir sur le monde physique

Un capteur convertit une grandeur physique — température, lumière, distance, humidité — en un signal électrique que le microcontrôleur peut lire. Un actionneur fait le chemin inverse : il transforme un ordre électrique en action (allumer une LED, faire tourner un moteur, ouvrir un relais, émettre un son). Les signaux sont de deux natures. Le signal numérique ne connaît que deux états, ou suit un protocole numérique précis. Le signal analogique est une tension continue : les entrées A0 à A5 de la Uno la mesurent grâce à un convertisseur analogique-numérique (CAN) de 10 bits, qui transforme une tension entre 0 et 5 V en un entier de 0 à 1023. Chaque pas vaut donc environ 5 V ÷ 1024 ≈ 4,9 mV : c'est la résolution de nos mesures analogiques.

Quelques capteurs à connaître. La photorésistance (LDR) voit sa résistance chuter quand la lumière augmente ; montée en diviseur de tension, elle donne une tension lisible sur une entrée analogique. Le TMP36 délivre une tension proportionnelle à la température : 10 mV par degré, avec un décalage de 500 mV, ce qui donne la formule $T = (V - 0{,}5) \times 100$. Les DHT11 et DHT22 mesurent température et humidité et communiquent par un protocole numérique sur un seul fil : une bibliothèque fait le travail, et l'on espace les lectures d'au moins une seconde pour laisser le capteur répondre. Le HC-SR04 mesure une distance par ultrasons : il émet un son inaudible et chronomètre son écho. Choisir un capteur, c'est comparer la grandeur mesurée, la plage, l'interface, le prix — et lire sa documentation (la « datasheet ») plutôt que de croire les promesses des vendeurs.

Côté actionneurs : la LED (avec sa résistance), le buzzer, le servomoteur dont on commande la position angulaire par un signal PWM (bibliothèque Servo), et le relais, un interrupteur commandé électriquement qui permet de piloter une charge plus puissante — pompe, lampe, ventilateur — en l'isolant de la carte. Règle de sécurité absolue : une broche de microcontrôleur ne fournit que quelques dizaines de milliampères. Brancher directement un moteur ou une pompe sur une broche, c'est risquer de détruire la carte ; on passe toujours par un module relais ou un transistor. Savoir ce qu'une broche peut et ne peut pas faire fait partie du métier.

L'essentiel
Capteur = physique → électrique ; actionneur = électrique → action. Le CAN 10 bits de la Uno convertit 0-5 V en 0-1023. Toute charge puissante (moteur, pompe) passe par un relais ou un transistor, jamais directement par une broche.

Exemple — Lire un capteur de température analogique TMP36

```
// Leçon 2 — Lire un capteur de température analogique TMP36
// Câblage : broche +Vs du TMP36 -> 5V de la carte
// broche GND du TMP36 -> GND de la carte
// broche Vout (centrale) -> entrée analogique A0

const int BROCHE_TMP36 = A0;

void setup() {
Serial.begin(9600); // ouvrir le moniteur série à 9600 bauds
}

void loop() {
int lecture = analogRead(BROCHE_TMP36); // 0 à 1023 (CAN 10 bits)
float tension = lecture * 5.0 / 1024.0; // conversion en volts
float temperature = (tension - 0.5) * 100.0; // formule du TMP36
Serial.print("Lecture brute : ");
Serial.print(lecture);
Serial.print(" -> ");
Serial.print(temperature, 1);
Serial.println(" degres C");
delay(1000);
}
```

À retenir

- Capteur = physique → électrique ; actionneur = électrique → action. Le signal peut être analogique (une tension) ou numérique (0/1 ou protocole).
- Le CAN 10 bits de la Uno transforme une tension de 0 à 5 V en un entier de 0 à 1023 : toute mesure analogique passe par une formule de conversion.
- Ne jamais alimenter un moteur ou une pompe depuis une broche : passer par un relais ou un transistor, la broche ne fournit que quelques dizaines de milliampères.

Exercice

Avec le montage de l'exemple (TMP36 sur A0), le moniteur série affiche « Lecture brute : 205 ». Calcule la tension correspondante puis la température, en détaillant chaque étape. À quelle température correspondrait une lecture brute de 0 ? Et de 1023 ? Que nous apprend ce calcul sur la plage réellement mesurable du montage ?

✔ Corrigé
Tension : V = 205 × 5,0 ÷ 1024 ≈ 1,001 V. Température : T = (1,001 − 0,5) × 100 ≈ 50,1 °C. Pour une lecture de 0 : V = 0 V, donc T = (0 − 0,5) × 100 = −50 °C. Pour 1023 : V = 1023 × 5,0 ÷ 1024 ≈ 4,995 V, donc T ≈ 449,5 °C. Ce dernier résultat est évidemment absurde pour une pièce : il illustre que la formule convertit fidèlement la tension, mais que la plage utile dépend du capteur lui-même (le TMP36 est conçu pour des températures ordinaires, largement en dessous) et que toute valeur extravagante doit d'abord faire suspecter un mauvais câblage — par exemple une broche Vout reliée au 5 V. Un bon réflexe d'ingénieur : vérifier la plausibilité avant de croire le chiffre.

Leçon 3 — Protocoles de communication pour l'IoT : initiation

Un objet connecté ne vaut que par les données qu'il échange. Choisir comment il communique suppose d'arbitrer entre trois critères souvent contradictoires : la portée (à quelle distance ?), le débit (combien de données ?) et la consommation (l'objet vit-il sur pile ?). Il faut aussi distinguer deux niveaux que les débutants confondent : le transport radio ou filaire, qui porte les octets (Wi-Fi, Bluetooth, LoRa, réseau cellulaire), et le protocole applicatif de messagerie, qui organise les échanges par-dessus (MQTT, CoAP, HTTP). Une station météo peut ainsi utiliser la radio Wi-Fi comme transport et MQTT comme messagerie : ce sont deux choix séparés.

Côté radio, le Wi-Fi offre un débit confortable et s'appuie sur l'infrastructure existante (la box du domicile ou du bureau), mais sa portée se compte en dizaines de mètres et il consomme beaucoup : adapté aux objets alimentés par le secteur. Le Bluetooth, et surtout sa version basse consommation BLE, couvre une dizaine de mètres et sert à dialoguer avec un téléphone proche : montres, capteurs portés, configuration d'appareils. Le LoRa transmet de très petits messages à plusieurs kilomètres en terrain dégagé, avec une consommation si faible qu'un capteur peut vivre des années sur pile : c'est le protocole des champs, des compteurs et des zones sans couverture. Le cellulaire adapté à l'IoT (NB-IoT, LTE-M) utilise le réseau des opérateurs, moyennant abonnement. Le Zigbee, enfin, organise un maillage de courte portée très utilisé en domotique.

Côté messagerie, MQTT est le protocole de référence de ce cours. Léger, conçu pour les réseaux peu fiables, il fonctionne au-dessus de TCP : les objets sont des clients qui se connectent à un serveur central appelé broker. Un client publie des messages sur des sujets (les topics), par exemple « iam/station/temperature » ; le broker les redistribue à tous les clients abonnés à ce sujet. Trois niveaux de qualité de service (QoS) règlent le compromis entre fiabilité et coût : 0 = au plus une fois, 1 = au moins une fois, 2 = exactement une fois. Le port standard est 1883, et 8883 quand les échanges sont chiffrés par TLS. Son concurrent CoAP vise les objets les plus contraints, au-dessus d'UDP, avec un modèle proche de HTTP ; HTTP lui-même reste possible, mais ses en-têtes lourds et ses connexions répétées pénalisent les petits objets sur pile.

L'essentiel
On choisit un protocole selon portée, débit et consommation. Radio (Wi-Fi, BLE, LoRa…) transporte les octets ; la messagerie (MQTT, CoAP) organise les échanges au-dessus. MQTT = publication/abonnement via un broker et des topics.

Exemple — Tableau comparatif des principaux protocoles IoT

```
Protocole Niveau Portee indicative Debit Consommation
--------- --------- ------------------- -------- ------------
Wi-Fi radio dizaines de metres eleve elevee
Bluetooth radio ~10 m (proximite) moyen faible (BLE)
LoRa radio plusieurs km tres bas tres faible
NB-IoT radio couverture operateur bas faible
Zigbee radio ~10-20 m, en maillage bas faible
MQTT messagerie (au-dessus de TCP) leger tres faible
CoAP messagerie (au-dessus de UDP) leger tres faible
HTTP messagerie (au-dessus de TCP) lourd plus elevee

Lecture : une ligne « radio » et une ligne « messagerie »
se combinent, par exemple LoRa + MQTT ou Wi-Fi + MQTT.
```

À retenir

- On choisit un protocole selon la portée, le débit et la consommation : Wi-Fi = débit, LoRa = distance et sobriété, Bluetooth = proximité avec un téléphone.
- MQTT n'est pas une radio : c'est un protocole de messagerie léger qui fonctionne au-dessus de TCP/IP, avec un broker, des topics et des abonnés.
- La QoS MQTT règle le compromis fiabilité/coût : 0 = au plus une fois, 1 = au moins une fois, 2 = exactement une fois.

Exercice

Pour chacun des trois projets suivants, choisis un transport radio et une messagerie, et justifie en deux phrases : (a) des capteurs d'humidité du sol répartis dans un champ situé à 3 km du village, sans Wi-Fi, alimentés par pile, avec un relevé toutes les 15 minutes ; (b) une station météo de balcon dont on consulte les mesures depuis le salon ; (c) un badge qui déverrouille une porte lorsque le téléphone de l'employé s'en approche.

✔ Corrigé
(a) LoRa + MQTT (via une passerelle LoRa-IP au village) : les relevés sont minuscules et espacés, la portée du LoRa couvre les 3 km et sa très faible consommation laisse les piles durer des saisons entières ; le Wi-Fi y est absent et trop gourmand. (b) Wi-Fi + MQTT : la box du domicile est à quelques mètres, l'alimentation secteur supprime le souci de consommation, et le broker permet de consulter les mesures depuis n'importe quelle application du salon. (c) Bluetooth BLE : le déverrouillage exige justement la proximité — quelques mètres — et le BLE est intégré à tous les téléphones ; une radio longue portée serait ici un défaut, pas une qualité, puisqu'elle validerait des badges éloignés.

Leçon 4 — Programmer un objet connecté simple

Nous assemblons maintenant les pièces : un capteur, une carte connectée et le protocole MQTT. La carte choisie est l'ESP32 : elle intègre le Wi-Fi et le Bluetooth, se programme avec la même IDE Arduino et les mêmes fonctions (pinMode, analogRead…), mais fonctionne en logique 3,3 V — ses broches ne tolèrent pas le 5 V, il faut donc vérifier la tension de chaque capteur avant de le brancher. L'architecture du système est celle de presque tous les objets connectés : le capteur mesure, la carte publie sur un topic du broker, et les applications (téléphone, tableau de bord, serveur) s'abonnent pour recevoir. L'objet suit un cycle immuable : se connecter au Wi-Fi, se connecter au broker, lire, publier, attendre, recommencer.

La bibliothèque PubSubClient fournit le client MQTT. Les appels essentiels sont peu nombreux : setServer(adresse, port) désigne le broker ; connect(identifiant) ouvre la session, avec un identifiant unique par objet ; publish(topic, message) envoie un message, le contenu étant une suite d'octets — on convertit donc les nombres en texte ; subscribe(topic) et une fonction de rappel permettent de recevoir des ordres, par exemple allumer une lampe à distance. L'organisation des topics fait partie de la conception : « iam/station/temperature » et « iam/station/humidite » permettent à un tableau de bord de s'abonner à toute la station d'un coup (« iam/station/# ») ou à une seule mesure.

Trois pièges attendent le débutant. D'abord, les identifiants : dans l'exemple, le mot de passe Wi-Fi est écrit dans le code à titre d'exercice de laboratoire, sur un réseau de test — en production, c'est une faute grave, et la leçon 6 expliquera pourquoi. Ensuite, le blocage : un long delay() empêche la fonction loop() d'appeler régulièrement le client MQTT, qui finit par être déconnecté ; on rythme donc les envois avec des délais raisonnables ou avec millis(). Enfin, la reconnexion : un objet réel doit détecter la perte du Wi-Fi ou du broker et se reconnecter tout seul, sans intervention humaine. Une cadence d'envoi adaptée à la grandeur mesurée — toutes les 10 secondes pour une température de pièce — évite d'inonder le réseau de messages inutiles.

L'essentiel
Un objet connecté répète : se connecter (Wi-Fi puis broker), lire le capteur, publier sur un topic, attendre. L'ESP32 (3,3 V, Wi-Fi intégré) et la bibliothèque PubSubClient suffisent pour un premier objet complet.

Exemple — Une station qui publie sa température en MQTT (ESP32 + DHT11)

```
// Leçon 4 — Station météo connectée : ESP32 + DHT11 + MQTT
// Exemple de LABORATOIRE : identifiants fictifs, réseau de test.
// En production, jamais de mot de passe en clair (voir leçon 6).
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

const char* SSID = "Wifi_Labo_IAM"; // à adapter au réseau
const char* MDP_WIFI = "motdepasse_exemple"; // fictif, pour l'exercice
const char* BROKER = "192.168.1.50"; // adresse du broker MQTT
const int BROCHE_DHT = 4; // DHT11 sur GPIO4

DHT dht(BROCHE_DHT, DHT11);
WiFiClient clientWifi;
PubSubClient mqtt(clientWifi);

void setup() {
Serial.begin(115200);
dht.begin();
WiFi.begin(SSID, MDP_WIFI);
while (WiFi.status() != WL_CONNECTED) { delay(500); }
mqtt.setServer(BROKER, 1883); // 1883 : MQTT non chiffré (labo)
}

void connecterMQTT() {
while (!mqtt.connected()) {
if (mqtt.connect("station-iam-01")) { /* connecté */ }
else { delay(2000); } // nouvel essai après 2 s
}
}

void loop() {
if (!mqtt.connected()) { connecterMQTT(); }
mqtt.loop(); // entretient la connexion
float t = dht.readTemperature();
if (!isnan(t)) { // lecture valide seulement
char message[8];
dtostrf(t, 4, 1, message); // 25.3 -> "25.3"
mqtt.publish("iam/station/temperature", message);
}
delay(10000); // un envoi toutes les 10 s
}
```

À retenir

• Un objet connecté suit toujours le même cycle : se connecter, lire le capteur, publier, attendre — puis recommencer en surveillant la connexion.

• L'ESP32 fonctionne en 3,3 V : vérifier la tension admissible par chaque capteur avant de le brancher.

• delay() bloque tout le programme : pour les objets connectés, on rythme les envois avec des délais raisonnables ou avec millis(), et on prévoit la reconnexion automatique.

Exercice

Modifie le sketch de l'exemple pour : (1) publier aussi l'humidité sur le topic « iam/station/humidite » ; (2) n'envoyer la température que si elle a changé d'au moins 0,5 °C depuis le dernier envoi, afin d'économiser la bande passante. Indique les variables à ajouter et les lignes modifiées (pas besoin de recopier tout le programme).

✔ Corrigé
Ajouter une variable globale float dernierеTemp = -100.0 (valeur impossible, pour forcer un premier envoi). Dans loop() : lire aussi float h = dht.readHumidity(). Pour l'humidité, publier à chaque cycle si !isnan(h), sur « iam/station/humidite » (conversion par dtostrf dans un second tampon). Pour la température, remplacer la publication inconditionnelle par : if (!isnan(t) && fabs(t - dernierеTemp) >= 0.5) { publier ; dernierеTemp = t ; }. La fonction fabs donne la valeur absolue d'un nombre réel : elle traite donc aussi bien une hausse qu'une baisse de 0,5 °C. Discussion : ce filtrage réduit le trafic, mais un tableau de bord ne peut plus distinguer « température stable » de « station en panne » ; en pratique, on combine souvent les deux règles — envoi si changement notable OU au moins une fois par minute, comme un signe de vie.

Leçon 5 — Collecte et affichage des données de capteurs

Une mesure isolée ne sert presque à rien : ce qui a de la valeur, c'est la série de mesures, horodatée et conservée, qui révèle une évolution (« le magasin chauffe chaque après-midi ») ou un incident (« la température du congélateur est montée à 3 h du matin »). La chaîne de traitement comporte cinq maillons : acquérir à cadence régulière, formater la mesure de façon structurée, transmettre, stocker, puis visualiser. Le format le plus simple est la ligne CSV — « seconde;temperature;humidite » — qui s'ouvre dans n'importe quel tableur pour tracer des courbes ; le format JSON, plus verbeux, se prête mieux aux échanges entre logiciels. Dans les deux cas, chaque enregistrement doit indiquer le moment, la valeur et l'unité.

L'affichage peut être local ou distant. Localement, le moniteur série sert au débogage, mais un vrai objet affiche sur place : l'écran LCD 1602 (2 lignes de 16 caractères) est le classique des montages Arduino. Sa version I2C est précieuse : grâce au bus I2C, seuls deux fils de données (SDA et SCL, c'est-à-dire A4 et A5 sur la Uno) suffisent, là où la version parallèle en réclame six ; l'adresse I2C du module est fréquemment 0x27, et la bibliothèque LiquidCrystal_I2C fait le reste. Un petit écran OLED ajoute des graphiques simples. À distance, l'affichage passe par les messages MQTT : un tableau de bord sur ordinateur ou téléphone s'abonne aux topics et dessine les courbes en temps réel.

La qualité des données se joue avant l'affichage. Chaque lecture doit être validée : pour un DHT, isnan() signale un échec de lecture (capteur mal branché ou sollicité trop vite) ; une tension qui dépasse la plage possible signale un fil débranché. On moyenne aussi plusieurs lectures rapprochées pour lisser le bruit électrique, et l'on choisit la cadence en fonction du phénomène : mesurer une température de pièce chaque seconde est inutile, chaque minute suffit. Enfin, gardons la tête froide : afficher « 25,37 °C » ne rend pas la mesure plus juste. Un capteur d'initiation comme le DHT11 donne des valeurs indicatives, excellentes pour apprendre et prototyper, mais ce n'est

pas un instrument de laboratoire certifié — et un bon technicien le dit clairement dans son rapport.

L'essentiel
Une donnée utile est horodatée, formatée (CSV ou JSON) et accompagnée de son unité. On valide chaque lecture (NaN, plage), on moyenne pour lisser, et l'affichage local (LCD I2C) comme distant (tableau de bord MQTT) se nourrissent des mêmes mesures vérifiées.

Exemple — Journal CSV sur le moniteur série et affichage LCD I2C

```
// Leçon 5 — DHT11 : journal CSV (série) + affichage LCD I2C
// LCD I2C : SDA -> A4, SCL -> A5 (Uno) ; DHT11 : données -> D2
#include <DHT.h>
#include <LiquidCrystal_I2C.h>

const int BROCHE_DHT = 2;
DHT dht(BROCHE_DHT, DHT11);
LiquidCrystal_I2C lcd(0x27, 16, 2); // adresse I2C fréquente : 0x27

void setup() {
Serial.begin(9600);
dht.begin();
lcd.init();
lcd.backlight();
Serial.println("seconde;temperature_C;humidite_pct");
}

void loop() {
float t = dht.readTemperature();
float h = dht.readHumidity();
if (isnan(t) || isnan(h)) {
Serial.println("Erreur de lecture du capteur");
delay(2000);
return; // on n'enregistre pas de fausse mesure
}
unsigned long seconde = millis() / 1000;
Serial.print(seconde); Serial.print(";");
Serial.print(t, 1); Serial.print(";");
Serial.println(h, 0);
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Temp: "); lcd.print(t, 1); lcd.print(" C");
lcd.setCursor(0, 1);
lcd.print("Hum : "); lcd.print(h, 0); lcd.print(" %");
delay(2000);
}
```

À retenir

- Une donnée utile est horodatée, formatée (CSV ou JSON) et accompagnée de son unité : sans cela, aucune analyse n'est possible ensuite.

• Toujours valider une lecture avant de l'enregistrer : un NaN ou une valeur hors plage signale un capteur mal branché ou défaillant.

• Le LCD en I2C n'utilise que deux broches de données (A4/A5 sur Uno) grâce au bus I2C, partagé avec d'autres modules.

Exercice

Le journal d'une station donne les lignes suivantes : « 0;24.0;60 », « 2;24.5;60 », « 4;Erreur de lecture du capteur », « 6;25.0;59 », « 8;25.5;60 ». (1) Calcule la température moyenne des mesures valides. (2) Explique pourquoi la ligne d'erreur ne doit pas entrer dans le calcul, et ce qu'elle signale probablement. (3) Le fichier mélange des lignes CSV et un message texte : pourquoi est-ce un problème pour un tableur, et que proposes-tu ?

✔ Corrigé
(1) Températures valides : 24,0 ; 24,5 ; 25,0 ; 25,5. Somme = 99,0 ; moyenne = 99,0 ÷ 4 = 24,75 °C. (2) La ligne d'erreur ne contient aucune mesure : la compter fausserait la moyenne (par exemple en la traitant comme 0 °C). Elle signale une lecture NaN du DHT — typiquement un capteur sollicité trop tôt après la précédente lecture, ou un fil de données mal enfiché ; si elle devient fréquente, il faut vérifier le câblage avant d'accuser le programme. (3) Un tableur importe le CSV en colonnes fixes : un message texte sans séparateurs casse la structure et peut décaler l'analyse. Bonne pratique : réserver le flux CSV aux mesures et signaler les erreurs ailleurs (un topic MQTT d'alerte, un fichier journal séparé ou une colonne « statut » valant OK/ERREUR dans chaque ligne).

Leçon 6 — Sécurité des objets connectés

Les objets connectés sont la cible préférée des attaquants, et pour de bonnes raisons : ils sont très nombreux, vendus au plus bas prix, rarement mis à jour, et souvent installés avec leurs réglages d'usine puis oubliés dans un coin. L'exemple historique est le botnet Mirai, en 2016 : ce logiciel malveillant a passé Internet au peigne fin à la recherche de caméras et d'objets encore protégés par leurs mots de passe par défaut — publiés dans les notices et identiques pour tous les exemplaires. Des centaines de milliers d'objets conquis ont ensuite été utilisés ensemble pour saturer de grandes plateformes par des attaques de déni de service. La leçon est limpide : un objet n'a pas besoin d'être intéressant pour être attaqué, il suffit qu'il soit prévisible.

Les défenses de base tiennent en une courte liste, valable pour l'usager comme pour le technicien. Changer tout mot de passe par défaut dès l'installation, et utiliser un mot de passe différent par appareil. Maintenir le micrologiciel (le firmware) à jour lorsque le fabricant publie des correctifs, et préférer dès l'achat les marques qui s'engagent à en fournir. Chiffrer les échanges : MQTT sur TLS (port 8883), pages d'administration en HTTPS, et authentification de chaque objet auprès du broker — un broker MQTT ouvert sans mot de passe sur Internet est une invitation. Ne jamais exposer un objet directement sur Internet par une redirection de port improvisée : l'objet reste derrière le routeur, idéalement sur un réseau Wi-Fi invité séparé de celui des ordinateurs.

Pour nos propres constructions, la sécurité se décide dès la conception. Chaque objet reçoit un identifiant et un secret uniques : si l'un est compromis ou perdu, on le révoque sans toucher aux autres. Aucun secret ne doit vivre dans un code publié — un dépôt Git public contenant le mot de passe du Wi-Fi ou la clé du broker est un secret déjà perdu, et l'exemple de la leçon 4 n'est tolérable que comme exercice sur un réseau de test isolé. On applique le moindre privilège (un capteur de température n'a pas à pouvoir publier sur le topic des commandes), on journalise les connexions pour repérer l'anormal, et l'on prévoit la procédure de mise à jour avant même le premier déploiement. La sécurité d'un objet connecté se joue autant dans ses réglages et son organisation que dans son code.

L'essentiel
Les failles reines de l'IoT : mots de passe par défaut (le botnet Mirai, 2016, s'est bâti dessus), firmware jamais mis à jour, échanges non chiffrés, objets exposés sur Internet. Réponses : mots de passe uniques, correctifs, TLS, authentification, secrets hors du code public.

Exemple — Fiche de durcissement d'une installation IoT (avant / après)

```
FICHE DE DURCISSEMENT — station météo du laboratoire

AVANT (à corriger) APRES (objectif)
------------------------------ ------------------------------
Caméra restée en admin/admin -> mot de passe unique et fort
Broker MQTT port 1883, -> TLS port 8883 + compte propre
sans mot de passe à chaque objet
Broker visible depuis -> broker uniquement sur le
Internet (port redirigé) réseau local / VPN
Mot de passe Wi-Fi dans un -> secrets hors du code, dépôt
dépôt Git public nettoyé, mot de passe changé
Firmware jamais mis à jour -> vérification des correctifs
à chaque maintenance

Règle d'or : tout réglage « d'usine » est un réglage
connu de tout le monde, donc de l'attaquant.
```

À retenir

- Le mot de passe par défaut est la première faille des objets connectés : le botnet Mirai (2016) s'est bâti presque uniquement dessus.
- Chiffrer (TLS), authentifier chaque objet, appliquer les correctifs du firmware et ne jamais exposer un objet directement sur Internet.
- Un secret écrit dans un code publié est un secret perdu : identifiants uniques par objet et aucune clé dans un dépôt public.

Exercice

Tu audites la serre connectée d'une école : (1) la caméra de surveillance est toujours en identifiant « admin » et mot de passe « admin » ; (2) le broker MQTT écoute sur le port 1883 sans aucun compte, et une redirection de port le rend joignable depuis

Internet ; (3) le sketch de l'ESP32 de la serre, avec le mot de passe du Wi-Fi de l'école en clair, est publié sur un dépôt Git public ; (4) aucun appareil n'a reçu de mise à jour depuis deux ans. Pour chaque point : nomme la faille, le risque concret, et la correction, puis classe les quatre corrections par priorité.

✔ Corrigé
(1) Mot de passe par défaut : n'importe qui peut se connecter à la caméra, regarder la cour et s'en servir comme tremplin vers le réseau. Correction : mot de passe unique et fort, compte administrateur renommé si possible. (2) Broker ouvert et exposé : un inconnu peut lire toutes les mesures, publier de fausses valeurs et même commander les actionneurs abonnés (arrosage forcé ou bloqué). Correction : supprimer la redirection de port, activer l'authentification, passer en TLS (8883) ou restreindre au réseau local. (3) Secret publié : le mot de passe Wi-Fi de toute l'école est compromis dès la publication — correction immédiate : changer le mot de passe Wi-Fi, retirer le secret du code et de l'historique du dépôt, adopter des identifiants par objet conservés hors du dépôt. (4) Absence de mises à jour : les failles connues des appareils restent béantes. Correction : inventorier les appareils, appliquer les correctifs disponibles, remplacer à terme ceux que le fabricant ne suit plus. Priorités : d'abord (3) et (2), qui donnent un accès immédiat depuis l'extérieur ; ensuite (1), accès tout aussi simple mais plus local ; enfin (4), qui relève le niveau général dans la durée.

Leçon 7 — Projet : station météo ou système d'arrosage connecté

Un projet embarqué se conduit comme un petit projet d'ingénierie. On commence par le besoin, formulé en une phrase mesurable : « relever température et humidité toutes les dix minutes, les afficher sur place et les publier en MQTT, avec une alerte si la température dépasse 40 °C », ou « arroser le jardin quand le sol est sec, jamais plus de cinq secondes d'affilée ». On en déduit la liste du matériel, puis l'on construit par étapes testées séparément : chaque capteur seul avec un mini-programme de test, puis l'affichage, puis la communication, puis seulement l'intégration complète. Brancher tout d'un coup et déboguer l'ensemble est la méthode la plus lente qui soit : quand rien ne marche, on ne sait pas qui accuser.

Les deux variantes du projet partagent la même ossature. La station météo s'appuie sur le DHT (température et humidité), affiche sur LCD et publie en MQTT ; on peut l'enrichir d'un capteur de pression ou d'une photorésistance qui date le lever du jour. Le système d'arrosage lit un capteur d'humidité du sol sur une entrée analogique et commande une pompe par un module relais ; ses règles de sécurité font partie du cahier des charges : durée d'arrosage maximale, pause minimale entre deux arrosages pour laisser l'eau pénétrer, et refus d'arroser sur une lecture incohérente (capteur arraché ou court-circuité). Dans les deux projets, l'étalonnage est l'étape que les étudiants négligent et que les professionnels soignent : comparer le DHT à un thermomètre de référence, et pour le sol, mesurer la valeur brute du capteur à l'air libre (sec) puis plongé dans l'eau, pour fixer les seuils du montage réel plutôt que des seuils copiés d'un tutoriel.

Le projet se termine par une documentation et une démonstration : schéma de câblage annoté, code commenté, extrait du journal de mesures, difficultés rencontrées et solutions adoptées, pistes d'amélioration (alimentation solaire, boîtier de protection, mode veille profonde pour ménager une batterie, tableau de bord web). C'est cette documentation, plus que le montage lui-même, qui distingue un bricolage d'un travail d'ingénieur : elle permet à un autre technicien de reproduire, dépanner et améliorer l'objet. La soutenance du projet se fait prototype en main et journal de mesures à l'appui — un objet connecté qui a réellement fonctionné plusieurs jours d'affilée vaut toutes les promesses.

L'essentiel
Conduire le projet comme un ingénieur : besoin mesurable, construction par étapes testées séparément, étalonnage des seuils sur le matériel réel, règles de sécurité pour les actionneurs, et documentation complète (schéma, code, journal de mesures).

Exemple — Squelette du système d'arrosage connecté (à compléter par l'envoi MQTT)

```
// Leçon 7 — Arrosage automatique : lecture du sol + pompe par relais
// Capteur d'humidité du sol -> A0 ; module relais -> D7 -> pompe.
// La plupart des modules relais sont actifs à l'état BAS (LOW) :
// vérifier la notice du module avant de brancher la pompe.

const int BROCHE_SOL = A0; // capteur d'humidité du sol
const int BROCHE_RELAIS = 7; // commande du relais (pompe)
const int SEUIL_SEC = 600; // valeur brute : A CALIBRER
const unsigned long ARROSAGE_MS = 5000; // 5 s maximum par arrosage
const unsigned long PAUSE_MS = 600000; // 10 min entre arrosages

bool arrosageDejaFait = false;
unsigned long dernierArrosage = 0;

void setup() {
pinMode(BROCHE_RELAIS, OUTPUT);
digitalWrite(BROCHE_RELAIS, HIGH); // relais au repos : pompe arrêtée
Serial.begin(9600);
}

void loop() {
int humidite = analogRead(BROCHE_SOL);
Serial.print("Humidite brute du sol : ");
Serial.println(humidite);
// Capteur résistif du TP : plus la valeur est haute, plus c'est sec.
// (Un capteur capacitif donne souvent la tendance inverse :
// d'où l'étalonnage systématique, voir le texte de la leçon.)
bool solSec = humidite > SEUIL_SEC;
bool pauseEcoulee = !arrosageDejaFait
|| (millis() - dernierArrosage >= PAUSE_MS);
if (solSec && pauseEcoulee) {
digitalWrite(BROCHE_RELAIS, LOW); // pompe en marche
delay(ARROSAGE_MS);
digitalWrite(BROCHE_RELAIS, HIGH); // arrêtée dans tous les cas
dernierArrosage = millis();
arrosageDejaFait = true;
}
delay(2000);
// A COMPLETER : publier l'humidité et les arrosages en MQTT (leçon 4).
}
```

À retenir

- On construit un objet connecté par étapes testées séparément (chaque capteur, puis l'affichage, puis la communication, puis l'intégration) : jamais tout brancher d'un coup.
- Tout seuil de capteur se calibre sur le matériel réel : mesurer la valeur « à sec » et la valeur « humide » avant de décider.
- Les règles de sécurité d'un actionneur (durée maximale, pause minimale, arrêt sur valeur incohérente) font partie du programme, pas des options.

Exercice

Pour la variante « station météo » du projet, rédige : (1) la liste complète du matériel ; (2) un plan de tests en quatre étapes, de la première LED au fonctionnement de bout

en bout ; (3) le critère chiffré qui te permettra de déclarer le projet réussi lors de la démonstration.

✔ Corrigé
(1) Matériel : une carte ESP32 (ou Uno + module Wi-Fi), un capteur DHT11 ou DHT22, un écran LCD 1602 I2C, une plaque d'essai et des fils de liaison, une résistance de tirage si le DHT n'est pas en module, un câble USB et une alimentation, un broker MQTT accessible (installé sur un ordinateur du laboratoire), un thermomètre de référence pour l'étalonnage, un ordinateur avec l'IDE Arduino et un tableur. (2) Plan de tests : étape 1, la carte seule — téléverser le clignotement de la leçon 1 et vérifier le moniteur série ; étape 2, le capteur seul — lire le DHT et comparer au thermomètre de référence, noter l'écart ; étape 3, l'affichage et la publication — LCD fonctionnel, messages visibles sur le broker avec un client de test ; étape 4, bout en bout — 24 heures de fonctionnement continu, journal CSV complet, aucune coupure non expliquée. (3) Critère de réussite (exemple) : sur 24 heures, au moins 95 % des relevés attendus (un toutes les 10 minutes, soit 144) sont présents dans le journal, et l'écart moyen avec le thermomètre de référence reste stable et documenté. Un critère chiffré, annoncé avant la démonstration, transforme une appréciation (« ça marche ») en résultat vérifiable.

Travaux dirigés (TD)

TD 1 — TD 1 — Diviseur de tension et télémètre à ultrasons

Première partie. Une photorésistance (LDR) est montée en diviseur de tension : elle relie le 5 V au point milieu lu par l'entrée A0, et une résistance fixe de 10 kΩ relie ce même point à la masse (GND). La tension au point milieu vaut V = 5 × 10 000 / (R_LDR + 10 000). (a) La LDR vaut 10 kΩ en lumière moyenne : calcule V et la lecture brute attendue sur A0. (b) Elle vaut 90 kΩ dans la pénombre : recalcule V et la lecture brute. (c) Explique en une phrase pourquoi ce montage transforme une variation de résistance en variation de tension lisible par la carte. Deuxième partie. Un HC-SR04 mesure la distance d'un mur : la durée aller-retour de l'écho est de 2 941 microsecondes. La vitesse du son dans l'air vaut environ 0,034 cm par microseconde. (a) Calcule la distance du mur. (b) Pourquoi divise-t-on la durée par deux dans la formule ? (c) Le programme affiche 850 cm alors que le mur est à un mètre : cite deux causes physiques ou matérielles possibles.

✔ Corrigé du TD 1
Première partie. (a) V = 5 × 10 000 / (10 000 + 10 000) = 5 × 0,5 = 2,5 V. Lecture brute = 2,5 / 5 × 1023 ≈ 512 : logique, le point milieu est à mi-tension. (b) V = 5 × 10 000 / (90 000 + 10 000) = 5 × 0,1 = 0,5 V. Lecture brute ≈ 0,5 / 5 × 1023 ≈ 102. (c) La LDR et la résistance fixe se partagent la tension d'alimentation au prorata de leurs valeurs : quand la résistance de la LDR varie avec la lumière, la part de tension au point milieu varie aussi, et le CAN de la carte la convertit en nombre. Deuxième partie. (a) Distance = 2 941 × 0,034 / 2 = 100 / 2 ≈ 50 cm. (b) Parce que la durée chronométrée couvre l'aller ET le retour du son : la distance du mur ne correspond qu'au trajet simple. (c) Causes possibles : un obstacle intermédiaire ou une surface inclinée qui renvoie un écho parasite plus tardif ; le capteur visait à côté du mur et l'écho provient d'un objet plus lointain ; une impulsion de déclenchement trop courte ou un câblage de la broche Echo sur une mauvaise entrée, qui fausse la mesure de durée. Réflexe : répéter la mesure à la main avec un mètre ruban sur trois distances connues avant d'accuser le code.

TD 2 — TD 2 — Choisir et nommer : protocoles et topics de trois projets

Un bureau d'études de Bamako reçoit trois demandes. Projet A : surveiller le niveau d'eau de 40 cuves réparties dans des villages du cercle de Kati, à plusieurs kilomètres les unes des autres, sans couverture Wi-Fi, avec des capteurs sur batterie remontant un relevé horaire. Projet B : piloter depuis le salon l'éclairage et la climatisation d'une maison déjà équipée d'une box Internet et d'une tablette murale. Projet C : un entrepôt veut enregistrer automatiquement la présence des employés quand leur carte approche à moins de deux mètres d'un lecteur de porte, et remonter les pointages au serveur du bureau le soir. Pour chaque projet : (1) choisis le transport radio principal et justifie par la portée, la consommation et l'infrastructure disponible ; (2) choisis la messagerie ; (3) pour le projet B, propose un plan de topics MQTT hiérarchisé couvrant l'éclairage du salon, la climatisation et la température du séjour.

✔ Corrigé du TD 2
Projet A : radio LoRa — seule option couvrant des kilomètres sans infrastructure, avec une consommation compatible avec des mois de batterie pour un relevé horaire minuscule ; messagerie MQTT au niveau de la passerelle qui regroupe les relevés et les transmet au serveur central. Projet B : Wi-Fi, déjà présent et alimenté par le secteur ; messagerie MQTT, dont le modèle publication/abonnement convient parfaitement à une tablette qui commande et reçoit les états. Projet C : la proximité exigée (moins de deux mètres) désigne le Bluetooth BLE pour la détection de la carte ; le transport des pointages stockés vers le serveur peut se faire en Wi-Fi le soir, ou en MQTT dès qu'une connexion est disponible — la messagerie n'impose pas la radio. Plan de topics pour B : une hiérarchie lieu/pièce/équipement/grandeur — « maison/salon/eclairage/commande » et « maison/salon/eclairage/etat », « maison/sejour/climatisation/commande » et « maison/sejour/climatisation/etat », « maison/sejour/temperature ». Séparer commande et état évite les boucles (un client qui s'abonne aux commandes ne se déclenche pas lui-même) et permet à la tablette de s'abonner à « maison/# » pour tout superviser, ou à un seul sous-arbre pour piloter une pièce.

Travaux pratiques (TP)

TP 1 — TP — Station météo complète : capter, afficher, journaliser, publier

Matériel : une carte Arduino Uno (ou ESP32), un capteur DHT11/DHT22, un écran LCD 1602 I2C, une plaque d'essai et des fils, un ordinateur avec l'IDE Arduino, un tableur, et (si disponible) un broker MQTT au laboratoire et un thermomètre de référence. Étape 1 — Capteurs seuls : câble le DHT (données sur D2) et affiche température et humidité sur le moniteur série toutes les 2 secondes. Vérifie le traitement des lectures NaN. Compare avec le thermomètre de référence et note l'écart. Étape 2 — Affichage : branche le LCD I2C (SDA sur A4, SCL sur A5) et affiche les deux mesures ; en cas d'erreur de lecture, l'écran doit afficher « Capteur ? » au lieu de valeurs inventées. Étape 3 — Journalisation : bascule l'affichage série en format CSV (« seconde;temperature;humidite »), enregistre 30 mesures consécutives, importe-les dans le tableur et trace la courbe de température. Calcule la moyenne et l'écart maximal sur la série. Étape 4 — Publication (si broker disponible) : reprends le sketch de la leçon 4 sur ESP32 et publie température et humidité sur des topics distincts, à une cadence de 10 secondes, avec gestion des NaN. Étape 5 — Compte rendu : une page comprenant le schéma de câblage, les résultats des étapes 1 à 4 (moyenne, écart au thermomètre, graphique) et trois difficultés rencontrées avec leurs solutions.

✔ Attendus du TP 1

Attendus et points de contrôle. Étape 1 : les valeurs doivent être stables à quelques dixièmes près quand on souffle doucement sur le capteur, l'humidité monte visiblement ; un écart constant avec le thermomètre (par exemple +1 °C) se note dans le compte rendu — c'est de l'étalonnage, pas une erreur de code. Étape 2 : l'adresse I2C du module est le plus souvent 0x27 ; si l'écran reste éteint, vérifier le rétroéclairage et le potentiomètre de contraste avant le code. Étape 3 : le CSV doit contenir exactement 30 lignes de données, sans ligne de texte parasite ; une moyenne calculée sur une série stable et un écart maximal faible valident l'ensemble. Étape 4 : les messages doivent être visibles avec un client de test abonné à « iam/station/# », et débrancher le Wi-Fi quelques secondes doit être suivi d'une reconnexion automatique. Barème indicatif : câblage et lecture correcte 30 %, affichage et erreurs gérées 20 %, journal CSV propre et graphique 25 %, publication MQTT 15 %, compte rendu 10 %. Difficultés classiques attendues : DHT lu trop rapidement (NaN), inversion SDA/SCL, fichier CSV pollué par les messages de débogage.

Exercices corrigés supplémentaires

Exercice 1

Ventilateur à hystérésis. Un ventilateur (via relais sur la broche 7) doit refroidir un local technique : démarrage quand la température atteint 30 °C, arrêt quand elle redescend à 28 °C. Écris la partie de loop() correspondante (le capteur étant lu dans la variable t), et explique pourquoi un seuil unique à 30 °C ferait « battre » le relais.

✔ Corrigé
Fragment : if (t >= 30.0) { digitalWrite(7, HIGH); } else if (t <= 28.0) { digitalWrite(7, LOW); } — entre 28 et 30 °C, on ne touche à rien : le ventilateur garde son état précédent, c'est la zone morte de l'hystérésis. Avec un seuil unique, la température réelle oscille naturellement autour de 30 °C (bruit de mesure, courants d'air) : le programme allumerait et éteindrait le relais à chaque lecture, plusieurs fois par minute. Conséquences : usure rapide des contacts du relais, claquements incessants, appels de courant répétés du moteur. L'hystérésis — deux seuils écartés — est le réglage standard de tous les thermostats, des réfrigérateurs aux régulateurs industriels.

Exercice 2

Chasse aux bogues. Ce sketch doit allumer une LED (broche 8) quand la lumière baisse (LDR sur A0, montée en diviseur comme au TD 1) et afficher la température d'un TMP36 (entrée A1), le tout sur une Arduino Uno. Il contient deux erreurs franches et un défaut de qualité : trouve-les et corrige-les. — void setup() { Serial.begin(9600); } void loop() { int lum = analogRead(A0); if (lum < 300) { digitalWrite(8, HIGH); } else { digitalWrite(8, LOW); } int brut = analogRead(A1); float tension = brut * 3.3 / 1024.0; float temp = (tension - 0.5) * 100.0; Serial.println(temp); delay(50); }

✔ Corrigé
Erreur 1 : la broche 8 n'est jamais déclarée en sortie — il manque pinMode(8, OUTPUT) dans setup(). Sans cette déclaration, la LED ne s'allumera pas correctement (elle peut tout au plus luire faiblement : piège classique, car le montage « a l'air » de réagir). Erreur 2 : la conversion utilise 3,3 V alors que la carte est une Uno dont le CAN est alimenté en 5 V : chaque température calculée est fausse. Par exemple, une lecture brute de 145 donnerait (145 × 3,3 ÷ 1024 − 0,5) × 100 ≈ −3,3 °C, au lieu d'environ 20,7 °C avec le bon calcul (145 × 5,0 ÷ 1024 = 0,707 V, puis (0,707 − 0,5) × 100). La référence de conversion doit toujours être la tension réelle du montage. Défaut de qualité : la température est affichée sans libellé ni unité — impossible, dans un journal, de savoir de quelle grandeur il s'agit. Version corrigée : setup() { pinMode(8, OUTPUT); Serial.begin(9600); } ; tension = brut * 5.0 / 1024.0 ; affichage : Serial.print("Temperature : "); Serial.print(temp, 1); Serial.println(" degres C");. Morale : en embarqué, les erreurs sont souvent silencieuses — aucun message d'erreur, juste un montage qui semble marcher avec des chiffres faux. Vérifier les ordres de grandeur fait partie du dépannage.

Exercice 3

Autonomie d'une station sur batterie. Une station météo ESP32 envoie un relevé toutes les 10 minutes et passe en veille profonde entre deux envois. Sur un cycle de 10 minutes, sa consommation moyenne mesurée est estimée à 5 mA. Sa batterie affiche une capacité de 2 000 mAh. (a) Calcule l'autonomie théorique en heures puis en jours. (b) Cite deux raisons pour lesquelles l'autonomie réelle sera inférieure. (c) Propose deux optimisations logicielles pour l'allonger.

✔ Corrigé
(a) Autonomie = capacité ÷ consommation moyenne = 2 000 mAh ÷ 5 mA = 400 heures, soit 400 ÷ 24 ≈ 16,7 jours. (b) L'autonomie réelle baisse car : la capacité annoncée d'une batterie est mesurée dans des conditions idéales (elle diminue avec la température, l'âge et le courant de décharge) ; les réveils du Wi-Fi consomment des pics de courant bien supérieurs à la moyenne et l'auto-décharge de la batterie s'ajoute pendant les périodes de veille. (c) Optimisations : espacer les envois (toutes les 30 minutes si le phénomène mesuré est lent) ; réduire le temps d'éveil — préparer la mesure avant d'allumer le Wi-Fi, publier, rendormir aussitôt ; grouper plusieurs mesures en un seul envoi ; baisser la puissance d'émission Wi-Fi si le point d'accès est proche. Toute optimisation sérieuse se valide par une mesure réelle sur 24 heures, pas seulement par le calcul.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Dans un sketch Arduino, à quoi sert la fonction setup() ?

A) À répéter le programme en boucle infinie

B) À configurer l'objet une seule fois, au démarrage

C) À lire les valeurs des capteurs analogiques

2. Le convertisseur analogique-numérique d'une Arduino Uno est de 10 bits. Quelles valeurs analogRead() peut-il renvoyer ?

A) De 0 à 1023

B) De 0 à 255

C) De 0 à 4095

3. Pour des capteurs agricoles répartis à plusieurs kilomètres, sur pile, avec de tout petits relevés espacés, le transport radio le plus adapté est :

A) Le Wi-Fi

B) Le Bluetooth

C) Le LoRa

4. Dans MQTT, quel est le rôle du broker ?

A) Il fabrique les capteurs et les cartes de prototypage

B) Il reçoit les messages publiés sur les topics et les redistribue aux abonnés

C) Il convertit les signaux analogiques en valeurs numériques

5. Un TMP36 branché sur A0 d'une Uno renvoie une lecture brute de 145. Quelle tension cela représente-t-il ?

A) Environ 0,21 V

B) Environ 1,45 V

C) Environ 0,71 V

6. Quel est le premier geste de sécurité à l'installation d'un objet connecté ?

A) Changer son mot de passe par défaut

B) Le peindre aux couleurs de l'entreprise

C) Désactiver définitivement son Wi-Fi

7. Pourquoi commande-t-on une pompe d'arrosage par un module relais plutôt que directement par une broche ?

A) Parce qu'une broche numérique ne peut pas produire d'état HIGH

B) Parce que la broche ne fournit que quelques dizaines de milliampères, insuffisants et dangereux pour la carte

C) Parce que le relais rend le programme plus rapide

8. La fonction readTemperature() d'un capteur DHT renvoie NaN. Que signifie ce résultat ?

A) La température vaut exactement 0 °C

B) Le capteur affiche la valeur spéciale 999

C) La lecture a échoué : il faut l'ignorer et vérifier cadence et câblage

9. Quelle est la tension logique d'une carte ESP32 programmée avec l'IDE Arduino ?

A) 3,3 V

B) 5 V

C) 12 V

10. En MQTT, la QoS de niveau 1 garantit qu'un message est livré :

A) Uniquement si le réseau est parfait

B) Au moins une fois (des doublons sont possibles)

C) Jamais plus d'une fois par semaine

Corrigés du QCM

1. Réponse B — setup() est exécutée une seule fois après la mise sous tension : on y déclare les modes des broches, on y démarre la liaison série et les bibliothèques. Le comportement répété vit dans loop().

2. Réponse A — 10 bits = 2^10 = 1 024 niveaux, numérotés de 0 à 1023, qui partagent la plage 0-5 V. 0-255 correspondrait à 8 bits, 0-4095 à 12 bits (le cas d'autres cartes).

3. Réponse C — Le LoRa couvre plusieurs kilomètres avec une consommation minuscule, au prix d'un débit très bas — exactement le profil de relevés agricoles espacés. Wi-Fi et Bluetooth sont des technologies de proximité.

4. Réponse B — Le broker est le serveur central du modèle publication/abonnement : les clients publient sur des topics, le broker transmet à tous les clients abonnés à ces topics. La conversion analogique-numérique est le rôle du CAN de la carte.

5. Réponse C — V = 145 × 5,0 ÷ 1024 ≈ 0,707 V, soit environ 0,71 V. La température correspondante vaut (0,707 − 0,5) × 100 ≈ 20,7 °C, une température de pièce tout à fait plausible.

6. Réponse A — Les mots de passe par défaut sont publics et identiques pour tous les exemplaires : c'est la faille qu'a exploitée le botnet Mirai en 2016. On le remplace par un mot de passe unique et fort dès l'installation.

7. Réponse B — Un moteur ou une pompe demande bien plus de courant qu'une broche ne peut en fournir : la brancher en direct risque de détruire le microcontrôleur. Le relais (ou un transistor) commute la puissance en gardant la commande isolée.

8. Réponse C — NaN (« Not a Number ») signale un échec de la lecture — capteur sollicité trop tôt, fil de données mal branché. Il se teste avec isnan() et ne doit jamais être enregistré ni publié comme une mesure.

9. Réponse A — L'ESP32 fonctionne en 3,3 V (contrairement à la Uno en 5 V) et ses broches ne tolèrent pas le 5 V : vérifier la fiche de chaque capteur avant de le brancher. Il intègre en échange le Wi-Fi et le Bluetooth.

10. Réponse B — QoS 0 = au plus une fois (perte possible), QoS 1 = au moins une fois (l'expéditeur réessaie jusqu'à l'accusé de réception, d'où des doublons possibles), QoS 2 = exactement une fois, au prix d'échanges supplémentaires.

Résumé — les notions importantes

• Un système embarqué est un ordinateur miniature dédié à une tâche, bâti autour d'un microcontrôleur (processeur, mémoires et entrées/sorties sur une puce) ; la Uno travaille en 5 V, l'ESP32 en 3,3 V avec Wi-Fi intégré.

• Capteurs et actionneurs dialoguent avec le monde physique ; le CAN 10 bits convertit les tensions 0-5 V en valeurs 0-1023, et chaque grandeur se calcule par une formule (TMP36, temps d'écho du HC-SR04).

• Les charges puissantes (moteurs, pompes) ne se branchent jamais sur une broche : relais ou transistor, avec des règles de sécurité dans le programme.

• Le choix d'un protocole IoT arbitre portée, débit et consommation : Wi-Fi pour le débit, Bluetooth pour la proximité, LoRa pour les kilomètres sur pile.

• MQTT organise les échanges au-dessus de TCP : un broker, des topics hiérarchisés, des clients qui publient et s'abonnent, et trois niveaux de QoS.

• Une donnée de capteur utile est validée (NaN, plausibilité), horodatée, formatée (CSV ou JSON), stockée et affichée ; tout seuil se calibre sur le matériel réel.

• La sécurité des objets connectés repose sur des gestes simples et non négociables : mots de passe par défaut changés et uniques, firmware à jour, échanges chiffrés, secrets hors du code public.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Allumer une LED par Arduino

Chaîne : Robotique Site · Langue : Français · Niveau : Débutant

Le tout premier montage Arduino : brancher une LED et l'allumer par le programme. Parfait pour valider son installation avant d'attaquer les capteurs de la leçon 2.

https://www.youtube.com/watch?v=GXOHjGLwgbg

Leçon 2 — TUTO ARDUINO : les CAPTEURS et MODULES les plus UTILES ! (+ tutorial branchement et programmation)

Chaîne : Bertrand . Tech · Langue : Français · Niveau : Débutant à intermédiaire

Tour d'horizon en français des capteurs et modules Arduino les plus courants, avec branchements et programmation pour chacun. Le segment consacré au DHT11 prolonge directement la leçon 2.

https://www.youtube.com/watch?v=fyvy0pY8AGc

Leçon 3 — Qu'est-ce que l'Internet des Objets (IoT) ? Le Guide Complet

Chaîne : ZoomIATech · Langue : Français · Niveau : Tous niveaux

Une vue d'ensemble de l'Internet des objets : architecture en couches, domaines d'application et questions de cybersécurité. À regarder pour replacer les protocoles de la leçon 3 dans le tableau général.

https://www.youtube.com/watch?v=_rnPQoL6Mys

Leçon 4 — Comment utiliser le protocole MQTT ? Introduction et application pratique

Chaîne : Electronique & Programmation · Langue : Français · Niveau : Intermédiaire

Introduction pratique à MQTT avec mise en œuvre concrète du modèle publication/abonnement. Le complément idéal du sketch ESP32 de la leçon 4.

https://www.youtube.com/watch?v=_cmBWyYHXmg

Leçon 6 — Domotique et sécurité en général.

Chaîne : PlaisirArduino · Langue : Français · Niveau : Intermédiaire

Les règles d'or pour sécuriser une installation domotique et des objets connectés : mots de passe, protocoles chiffrés et bonnes pratiques. En écho direct à la fiche de durcissement de la leçon 6.

https://www.youtube.com/watch?v=ZYoGfIYLPXg

Sources et références

- Documentation officielle Arduino : référence du langage, tutoriels et guides de démarrage — arduino.cc et docs.arduino.cc.
- Norme MQTT : spécifications OASIS MQTT 3.1.1 et MQTT 5.0 et documentation du projet — mqtt.org ; serveur Mosquitto — mosquitto.org.
- Documentation Espressif pour l'ESP32 (fiches techniques, guides de programmation) — docs.espressif.com ; bibliothèque cliente PubSubClient pour Arduino.
- Documentation LoRa et LoRaWAN de la LoRa Alliance — lora-alliance.org.
- Recommandations de sécurité pour les objets connectés publiées par l'ANSSI (Agence nationale de la sécurité des systèmes d'information) — cyber.gouv.fr.

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !